

Kapitel 2

Standarddatentypen, Konstanten und Variablen

2.1 Interne Darstellungen

Heutige Computer verfügen über Schaltelemente, die zwei stabile Zustände annehmen können. Identifiziert man diese beiden Zustände mit den Zahlen 0 und 1, so kann man *Binärzahlen*, d.h. Zahlen im Stellenwertsystem zur Basis 2, in einem Rechner speichern und verarbeiten.

Die *Dezimalzahlen*, mit denen zu rechnen wir gewohnt sind, lassen sich problemlos und eindeutig auch als Binärzahlen darstellen. Sehen wir uns ein Beispiel dafür an:

$$\begin{aligned} 109_d &= 1 \cdot 10^2 + 0 \cdot 10^1 + 9 \cdot 10^0 \\ &= 64 + 32 + 8 + 4 + 1 \\ &= 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1101101_b \end{aligned}$$

Die dreistellige Dezimalzahl 109 ergibt bereits eine siebenstellige Binärzahl; die dreistellige Dezimalzahl 999 würde bereits eine zehnstellige Binärzahl ergeben. Da Zahlen mit so vielen Ziffern nur sehr schlecht zu lesen sind, zumal wenn sie wie Binärzahlen nur aus Nullen und Einsen bestehen, sind im Zusammenhang mit Rechnern andere Darstellungen üblich, nämlich *Oktalzahlen* (Basis 8; Ziffern: 0, ..., 7) und *Hexadezimalzahlen* (Basis 16; Ziffern: 0, ..., 9, A, ..., F). Die Umwandlung zwischen Binär- und Oktaldarstellung bzw. zwischen Binär- und Hexadezimaldarstellung ist besonders einfach: Jede Oktalziffer entspricht gerade einer Gruppe von 3 Binärziffern, jede Hexadezimalziffer gerade einer Gruppe von 4 Binärziffern. Wir sehen uns auch das an einem Beispiel an:

$$\begin{aligned} 109_d &= 1101101_b \\ &= 1_b \cdot 8^2 + 101_b \cdot 8^1 + 101_b \cdot 8^0 \\ &= 1_d \cdot 8^2 + 5_d \cdot 8^1 + 5_d \cdot 8^0 \\ &= 155_o \\ \\ 109_d &= 1101101_b \\ &= 110_b \cdot 16^1 + 1101_b \cdot 16^0 \\ &= 6_d \cdot 16^1 + 13_d \cdot 16^0 \\ &= 6D_h \end{aligned}$$

Verarbeitet werden ohne weiteres nie einzelne Bits, sondern stets Gruppen. Eine gebräuchliche Gruppe ist das *Byte*, das aus 8 Bits besteht.

2.2 Wertebereiche

Wir haben eben bereits gesehen, wie man Zahlen (genauer: nichtnegative ganze Zahlen) in einem Rechner darstellen kann. Diese Zahlen reichen aber nicht aus: Vielfach möchte man auch Zeichen oder reelle Zahlen verarbeiten. Dem tragen die *Standardtypen* der Programmiersprachen Rechnung. Letztlich sind sie nur Vorschriften, Bitfolgen bestimmter Länge in bestimmter Weise zu interpretieren.

Die Standardtypen, die C kennt, werden uns in den nächsten Abschnitten beschäftigen. Ein generelles Problem dabei ist, dass auf der einen Seite der Wertebereich eines Zahlentyps direkt von der Anzahl der Bits abhängt, die für ihn verwendet werden, dass auf der anderen Seite C aber unabhängig von bestimmter Hardware definiert werden sollte.

Wir sehen uns dafür ein Beispiel an: Die am weitesten verbreitete Maßeinheit ist das bereits angesprochene Byte. In ihm kann man die Zahlen $00000000_b = 0_d$ bis $11111111_b = 255_d$ speichern – ein für die Praxis viel zu kleiner Wertebereich. Zwei Bytes ergäben den Wertebereich 0 bis 65535 – auch noch nicht übermäßig viel.

Will man neben den positiven auch negative Zahlen darstellen, so benötigt man ein Bit für das Vorzeichen und erhält dann für Bytes den Wertebereich -127 (bzw. -128) bis $+127$ bzw. für zwei Bytes den Wertebereich -32767 (bzw. -32768) bis $+32767$.

Man hätte nun bei der Definition von C sagen können: In den Programmen können ganzzahlige Werte mit 1, 2 oder 4 Bytes verarbeitet werden. Das wäre aber keine zweckmäßige Definition gewesen, da sie eine optimale Implementation auf vielen Rechnern verhindert hätte:

- Nicht jeder Rechner kennt Bytes – viele Großrechner haben eine *Wortstruktur*, wobei jedes Wort aus 64 oder mehr Bits besteht, und können auf kürzere Bitfolgen gar nicht ohne weiteres zugreifen.
- Auch bei manchem Rechner, der Bytes kennt, ist es für das Rechnen in landläufigem Sinne günstiger, wenn statt einzelner Bytes Gruppen von zwei oder mehr Bytes verwendet werden.

Der ANSI-Standard für C trägt dem Rechnung, indem er keine festen Wertebereiche, sondern nur *Mindestschränken* für die Wertebereiche vorschreibt, und indem er außerdem vorschreibt, dass für die verschiedenen Typen bestimmte *benannte Konstanten* in bestimmten Standard-Headerdateien verfügbar sein müssen, die die konkreten, implementationsspezifischen Werte wiedergeben. Diese Headerdateien sind `limits.h` für die Wertebereiche der ganzzahligen Typen und `float.h` für die Gleitkommatypen.

2.3 Ganzzahlige Typen

Beginnen wir nun konkret mit dem Typ, der bislang exemplarisch behandelt wurde, nämlich dem Typ für die Beschreibung ganzer Zahlen (*integer*). Dieser Typ wird in C mit dem Schlüsselwort `int` bezeichnet.

Genau genommen bezeichnet `int` eine *Gruppe von Typen*, wie es unsere Vorüberlegungen auch bereits nahelegen:

- Neben `int` selbst gibt es `short int` (oder kurz: `short`) mit möglicherweise kleinerem Wertebereich und `long int` (oder kurz: `long`) mit möglicherweise größerem Wertebereich.

- Man hat für alle drei Wertebereiche die Wahl zwischen vorzeichenbehafteten (Schlüsselwort **signed**) und vorzeichenlosen (Schlüsselwort **unsigned**) Zahlen. Gibt man keines der beiden Schlüsselworte an, werden die Zahlen als vorzeichenbehaftet betrachtet.

In Tabelle 2.1 finden Sie die möglichen, alternativen Bezeichnungen für die verschiedenen Ganzzahl-Typen und die Mindestschranken, die der Standard für sie vorschreibt.

Typ	Minimum	Maximum
signed char	≤ -127	≥ 127
unsigned char	$= 0$	≥ 255
signed short int signed short short int short	≤ -32767	≥ 32767
unsigned short int unsigned short	$= 0$	≥ 65535
signed int signed int	≤ -32767	≥ 32767
unsigned int unsigned	$= 0$	≥ 65535
signed long int signed long long int long	≤ -2147483647	≥ 2147483647
unsigned long int unsigned long	$= 0$	≥ 4294967295

Tabelle 2.1: Vorgeschriebene Wertebereiche für Ganzzahltypen

Die im Standard festgelegten Namen der Schranken-Konstanten in `limits.h` sind

- `SHRT_MIN`, `SHRT_MAX` und `USHRT_MAX` für die **short**-Typen,
- `INT_MIN`, `INT_MAX` und `UINT_MAX` für die **int**-Typen und
- `LONG_MIN`, `LONG_MAX` und `ULONG_MAX` für die **long**-Typen.

Im Quellcode kann man Konstanten mit ganzzahligen Typen wahlweise dezimal, oktal oder hexadezimal schreiben:

- Dezimale Konstanten beginnen mit einer Ziffer ungleich 0.
- Oktale Konstanten beginnen mit einer 0.
- Hexadezimale Konstanten beginnen mit `0x` oder `0X`; dieser Kennung muss mindestens eine hexadezimale Ziffer folgen.

Die Dezimalzahl 109, für die wir oben die Umwandlung betrachtet haben, können wir in einem Programm also wahlweise so schreiben:

```
109          dezimal
0155        oktal
0x6D        hexadezimal
0X6D        hexadezimal
```

Eine dezimal geschriebene Konstante erhält den ersten möglichen Typ aus **int**, **long** und **unsigned long**; eine oktal oder hexadezimal geschriebene Konstante erhält den ersten

möglichen Typ aus `int`, `unsigned int` und `unsigned long`. Der Programmierer kann den Typ aber auch explizit festlegen, indem er einer Konstanten die Buchstaben `u` oder `U` für `unsigned`-Typen und/oder `l` oder `L` für `long`-Typen nachstellt. So ist

```
109lu
```

die dezimale Darstellung der Dezimalzahl 109 im Typ `unsigned long`.

Negative Konstanten kennt C nicht! Vorzeichen werden vielmehr stets als einstellige (unäre) Operatoren interpretiert. Da es sich bei Konstanten mit negativem Vorzeichenoperator jedoch um einen konstanten Ausdruck handelt, werden diese trotzdem vom Compiler und nicht zur Laufzeit berechnet. Aber darauf werden wir erst später zurückkommen.

2.4 Der Typ `char`

Zeichen (*character*) werden von C als Teilmenge der ganzen Zahlen behandelt; den Typ bezeichnet das Schlüsselwort `char`. Das funktioniert so: Die verfügbaren Zeichen werden durchnummeriert, beginnend bei 0. Im Rechner werden die Zeichen dargestellt, indem diese *Ordnungszahlen* gespeichert werden. Letztlich geht jede Programmiersprache so vor, nur ist in der Regel die Koppelung von Zahlen und Zeichen nicht so eng.

Wieder ergeben sich Probleme aus der Tatsache, dass C auf möglichst allen Rechnern implementiert werden können soll, auch wenn die Rechner mit unterschiedlichen Zeichensätzen arbeiten. So kann nur vorgeschrieben werden, dass bestimmte Zeichen vorhanden sein müssen, nämlich

- die 26 Großbuchstaben des (englischen) Alphabets,
- die 26 Kleinbuchstaben des (englischen) Alphabets,
- die 10 (Dezimal-)Ziffern,
- 30 druckbare Sonderzeichen wie Leerzeichen, Plus und Minus, Punkt, Komma, Klammern, usw., und
- 7 Steuerzeichen zur Steuerung von Ein-/Ausgabegeräten.

Man beachte: Die deutschen Umlaute und das β gehören *nicht* zum Zeichensatz von ANSI-C! Weitestgehend entsprechen diese Anforderungen des Standard der Realität des *ASCII-Code* (*American Standard Code for Information Interchange*), der insgesamt 128 Zeichen umfasst:

- Die Zeichen mit den Ordnungszahlen 0 bis 31 sowie das Zeichen mit der Ordnungszahl 127 sind Steuerzeichen.
- Die Zeichen mit den Ordnungszahlen 32 bis 126 sind druckbar.

Dass an dieser Stelle keine Tabelle mit den Ordnungszahlen der verschiedenen ASCII-Zeichen zu finden ist, liegt daran, dass man sich die Ordnungszahlen der Zeichen bewusst nicht merken sollte. Durch die Zeichenkonstanten hat man in C jederzeit Zugriff auf die Ordnungszahl eines Zeichens, ohne den Wert selbst kennen zu müssen.

Zur internen Speicherung von Zeichen wird das Byte verwendet, das seinerseits über mindestens 8 Bits verfügen muss.

Obwohl ein Vorzeichen bei Zeichen nicht plausibel erscheint, unterscheidet man zwischen `signed` und `unsigned`, damit der Typ `char` kompatibel zu den `int`-Typen bleibt, deren Teilmenge er ja ist. Die Wertebereiche der beiden `char`-Typen wurden bereits in Tabelle 2.1 mit aufgelistet.

Die konkreten implementationsspezifischen Werte enthält `limits.h` in Form der Konstanten

- `CHAR_MIN` und `CHAR_MAX` für den Typ `char` ohne Zusatz,
- `SCHAR_MIN` und `SCHAR_MAX` für den Typ `char` mit dem Zusatz `signed` und
- `UCHAR_MAX` für den Typ `char` mit dem Zusatz `unsigned`.

Aus der Existenz der verschiedenen Werte kann man schließen, dass der Typ `char`, anders als die `int`-Typen, je nach Implementation vorzeichenlos oder vorzeichenbehaftet sein kann, wenn das nicht explizit durch `unsigned` oder `signed` festgelegt wird.

Zeichenkonstanten sind einzelne Zeichen, eingeschlossen in Apostrophe, etwa

```
'w'
```

Eine Zeichenkonstante repräsentiert die Ordnungszahl ihres Zeichens – welche das ist, hängt vom Zeichensatz ab, den der Rechner verwendet. Dabei ist eines noch wichtig zu erwähnen, weil es oft vergessen wird: Die Zeichenkonstante `'0'` repräsentiert in der Regel *nicht* die Ordnungszahl 0, die Zeichenkonstante `'1'` in der Regel *nicht* die Ordnungszahl 1, usw. Gesehen haben wir das für den ASCII-Code im Grunde genommen bereits: Die Ordnungszahlen der druckbaren Zeichen beginnen in ihm erst bei 32!

Einige Zeichen können nicht ohne weiteres angegeben werden. Dafür gibt es zwei Gründe:

- Einige druckbare Zeichen haben im Quellcode bestimmte Bedeutung. Von diesen haben wir den Apostroph bereits kennengelernt, der (u.a.) Zeichenkonstanten begrenzt.
- Die Steuerzeichen für Ein-/Ausgabegeräte, die C definiert, stehen auf der Tastatur nicht zur Verfügung.

Für elf solche Zeichen stellt C *Escapesequenzen* bereit. Tabelle 2.2 listet diese auf.

Zeichen	Beschreibung
<code>\a</code>	Piepen (wird nicht von allen Ausgabegeräten unterstützt)
<code>\b</code>	Backspace
<code>\f</code>	Seitenvorschub (wird nur von Druckern beachtet)
<code>\n</code>	Zeilenvorschub
<code>\r</code>	Positionierung auf Zeilenanfang
<code>\t</code>	Horizontaler Tabulator
<code>\v</code>	Vertikaler Tabulator (wird nur von Druckern beachtet)
<code>\'</code>	Apostroph, der nicht zur Begrenzung einer Zeichenkonstante dient
<code>\"</code>	Gänsefüßchen, die nicht zur Begrenzung einer Stringkonstante dienen
<code>\\</code>	Backslash
<code>\?</code>	Fragezeichen. Nur relevant, wenn man zwei unmittelbar aufeinanderfolgende Fragezeichen in einer Zeichenkettenkonstanten verwenden möchte. Hintergrund ist die selten benötigte Möglichkeit, Sonderzeichen durch eine Kombination aus zwei Fragezeichen und einem weiteren Zeichen darzustellen („Trigraphen“).

Tabelle 2.2: Escapesequenzen

2.5 Zeichenkettenkonstanten

Eine *Zeichenkettenkonstante*, der Kürze wegen gerne mit dem englischen Begriff *String* bezeichnet, ist eine Folge von beliebig vielen Zeichen, die in Gänsefüßchen eingeschlossen ist. Wir haben einen solchen String bereits im einführenden Beispiel verwendet. Strings, die nur durch „white spaces“ getrennt sind, werden vom Präprozessor zu einem einzigen String zusammengefügt.

```
printf("Dieser String ist zu lang, um ihn in einer "
      "einzelnen Zeile unterzubringen.\n");
```

Der Backslash `\` ist in einer Zeichenkettenkonstante nur in Verbindung mit einer Escapesequenz erlaubt. Zeilenumbrüche sind gar nicht erlaubt. Soll ein Zeilenumbruch in der Ausgabe erfolgen, so ist die Escapesequenz `\n` zu verwenden. Da Gänsefüßchen zur Begrenzung von Strings dienen, können diese ebenfalls nur als Escapesequenz `"` innerhalb eines Strings auftauchen.

Achtung: `'x'` bedeutet nicht dasselbe wie `"x"`! Darauf werden wir in anderem Zusammenhang noch zurückkommen.

2.6 Gleitkommatypen

Zum numerischen Rechnen benötigt man *Gleitkommazahlen*. C stellt diese mit den Typen `float`, `double` und `long double` zur Verfügung. Alle drei Typen müssen mindestens den Wertebereich

$$[-10^{+37}, -10^{-37}] \cup \{0\} \cup [+10^{-37}, +10^{+37}]$$

besitzen. Zur Beschreibung von Gleitkommazahlen reicht die Angabe des Wertebereichs allerdings nicht aus; ebenso benötigt man die Angabe der *Dichte* der Zahlen. Sie kann durch den Abstand der Zahl 1 von der nächstgelegenen größeren Zahl beschrieben werden. Der Standard verlangt die minimalen Dichten

- 10^{-5} beim Typ `float` und
- 10^{-9} bei den Typen `double` und `long double`.

Das entspricht einer Genauigkeit von 6 bzw. 10 dezimal-Stellen. Während sich diese Angaben auf die dezimale Darstellung beziehen, werden Gleitkommazahlen intern binär verarbeitet. Bei der Umwandlung von der einen in die andere Form treten zwangsläufig Rundungsfehler auf. Die Mindestanzahl an Dezimalstellen, die bei Umwandlung in Binärdarstellung und Rückumwandlung erhalten bleibt, wird durch die Anzahl der signifikanten Stellen angegeben. Auch dafür macht der Standard eine Vorgabe.

Die konkreten Werte für die Wertebereiche der Gleitkommatypen findet man in der Datei `float.h`:

- Wertebereich: `FLT_MAX`, `DBL_MAX`, `LDBL_MAX`
- Dichte: `FLT_EPSILON`, `DBL_EPSILON`, `LDBL_EPSILON`
- Anzahl der signifikanten Stellen: `FLT_DIG`, `DBL_DIG`, `LDBL_DIG`.

Im Programm hat man prinzipiell zwei Möglichkeiten der Darstellung:

- Man kann einen Dezimalpunkt schreiben, der vor, inmitten oder hinter der Ziffernfolge stehen kann.
- Man kann einen Exponententeil angeben, der mit dem Kennbuchstaben `e` oder `E` beginnt und danach den Exponenten zur Basis 10 als ganze Zahl mit oder ohne Vorzeichen aufweist.

Außerdem kann man diese beiden Möglichkeiten auch kombinieren.

Einige Beispiele:

.1	0.1	1e-1	1.0E-1
1.	1.0	1e0	1.0E+0
10.	10.0	1e1	1.0E+1

Ohne weiteres besitzen Gleitkommakonstanten den Typ `double`. Mit den nachgestellten Kennbuchstaben `f` oder `F` bzw. `l` oder `L` können aber auch Konstanten mit dem Typ `float` bzw. `long double` angegeben werden.

2.7 Aufzählungskonstanten

Aufzählungskonstanten sind Konstanten mit ganzzahligem Wert, die durch eine sogenannte `enum`-Deklaration einen Namen erhalten, mit dem sie im weiteren angesprochen werden können.

```
enum [name] { konstantenliste };
```

Dabei:

- Das Schlüsselwort `enum` leitet die Deklaration ein.
- Optional kann der Aufzählungstyp benannt werden. Die eckigen Klammern sollen andeuten, dass der Name *name* optional ist; sie dürfen in keinem Falle mitgeschrieben werden.
- Die Liste der Konstanten wird in geschweifte Klammern eingeschlossen. Die Elemente der Liste werden voneinander jeweils durch ein Komma getrennt. Es ist üblich, wenn auch nicht notwendig, für die Namen der Konstanten ausschließlich Großbuchstaben zu verwenden, was der Lesbarkeit des Programms dienen soll.
- Die Vereinbarung wird durch ein Semikolon abgeschlossen.

Beispiel

```
enum farbe { ROT, GELB, BLAU, GRUEN };
```

Dem ersten Namen in einer `enum`-Liste wird der Wert 0 zugeordnet, allen weiteren der um eins erhöhte Wert des Vorgängers.

Man kann die Werte jedoch auch explizit in der Form `= wert` angeben. Dann wird für Namen, für die kein Wert angegeben ist, aufsteigend weitergezählt. Beispiel:

```
enum escapes { BELL = '\a' , NEWLINE = '\n' };
enum farbe { ROT = 1, GELB = 4, BLAU, GRUEN = 2 };
```

Hier erhält `BLAU` den Wert 5, während allen anderen Namen die angegebenen Werte zugeordnet werden.

Alle Namen in einer oder verschiedenen Aufzählungsdeklarationen im selben Gültigkeitsbereich müssen sich unterscheiden. Die Werte, die verschiedenen Namen zugeordnet werden, auch innerhalb einer einzelnen Aufzählungsdeklaration, brauchen nicht verschieden zu sein. Auch negative Werte können vergeben werden. Einzige Restriktion: Die Werte müssen ganzzahlig sein.

2.8 Benannte Konstanten

Aufzählungskonstanten sind auf ganzzahlige Werte beschränkt. Aber auch für Gleitkommawerte muss man eine Möglichkeit haben, symbolische Namen zu vergeben. Erste Beispiele sind mit `FLT_MAX` oder `DBL_EPSILON` schon genannt worden. Generell sind benannte

Konstanten für den Programmierer im Alltag wichtig, um „magische Konstanten“ zu vermeiden.

Betrachten wir als Beispiel ein Programm, in dem Verkaufspreise berechnet werden. Stand in ihm an allen Stellen, an denen Mehrwertsteuer berechnet wird, explizit die Konstante 14 bzw. 1.14, so gab es zum 1.1.1993 mit der Erhöhung der Mehrwertsteuer viel Arbeit für die Programmierer: Diese Konstanten mussten alle ersetzt werden! Ein besonderes Problem dabei ist: Für jede 14 bzw. 1.14 musste separat entschieden werden, ob die 14 bzw. 1.14 die Mehrwertsteuer war oder eine andere Größe bezeichnete, zum Beispiel den Verschnitt des Materials, die nicht verändert werden durfte.

Mit Hilfe der Präprozessor-Direktive

```
#define name ersatztext
```

kann man solchen Konstanten sinnvolle Namen geben, etwa

```
#define MWST 14.0
```

In allen nachfolgenden Zeilen wird an allen Stellen, an denen *name* nicht innerhalb eines String oder als Teil eines anderen Namens steht, *name* durch *ersatztext* ersetzt. Wie bei den Aufzählungskonstanten ist es üblich, für die Namen ausschließlich Großbuchstaben zu verwenden, auch wenn der Standard das nicht vorschreibt.

Aufzählungskonstanten und durch `#define` benannte Konstanten haben manches gemeinsam. Es gibt aber (natürlich) auch Unterschiede:

- Bei `#define` muss man die Werte stets explizit angeben, während man bei Aufzählungskonstanten den Compiler zählen lassen kann. Das ist insbesondere dann interessant, wenn man nur erreichen will, dass die Konstanten verschiedene Werte repräsentieren und ggf. eine bestimmte Größenrelation zwischen ihnen besteht, die die Werte selbst aber unwesentlich sind.
- Aufzählungskonstanten können nur ganzzahlige Werte zugeordnet werden, während bei `#define` beliebige Werte – und wie wir später sehen werden nicht nur Werte – angegeben werden können.

Erweitern wir dazu unser Beispiel der Mehrwertsteuer noch etwas: Benötigt man im Programm neben dem vollen Steuersatz auch den ermäßigten, so könnte (und sollte) man schreiben

```
#define MWST 14.0
#define MWST_ERM (MWST / 2)
```

statt an allen entsprechenden Stellen explizit `MWST / 2` hinzuschreiben. Der ermäßigte Steuersatz lag bislang zwar immer bei der Hälfte des vollen – nur war das eher Zufall und Gewohnheit. Der Vorteil dieser Formulierung: Für die Änderung zum 1.1.1993 brauchten nur diese beiden Zeilen durch

```
#define MWST 15.0
#define MWST_ERM 7.0
```

ersetzt und danach das Programm neu übersetzt und gebunden zu werden.

2.9 Variablen

Konstanten verändern ihren Wert zur Laufzeit eines Programms nicht. Sie besitzen außerdem nur ihren Wert und Typ, nicht jedoch einen Speicherplatz ; zumindest nicht in der Theorie, in der Praxis sieht das gelegentlich anders aus.

Variablen besitzen dagegen einen Speicherplatz und Typ und in der Regel auch einen Wert, nämlich den, der an diesem Speicherplatz steht. Es ist jederzeit möglich, diesen Wert zu ändern. Solange der Wert einer Variablen nicht gesetzt wurde, braucht sie keinen wohldefinierten Wert zu besitzen.

In C müssen alle Variablen vereinbart werden, bevor sie verwendet werden können. Gewöhnlich geschieht dieses am Anfang einer Funktion. Die Vereinbarung von Variablen hat die Form

```
[speicherklasse] typ namenliste ;
```

Dabei:

- Die Angabe eines Speicherklassen-Attributs ist optional; darauf werden wir später zurückkommen.
- *typ* ist eines der Schlüsselwörter oder eine der Schlüsselwort-Kombinationen, die wir bereits kennengelernt haben, oder auch ein Aufzählungstyp. Auf weitere Möglichkeiten werden wir später zurückkommen.
- Die *namenliste* enthält, durch je ein Komma voneinander getrennt, die Namen der zu vereinbarenden Variablen.
- Die Vereinbarung wird durch ein Semikolon abgeschlossen.

Beispiele:

```
int i;  
double a, b, c;  
enum farbe kreide;
```

Solch eine Vereinbarung hat zwei Funktionen:

- Zum einen wird der Compiler veranlasst, Speicherplatz für die Variablen bereitzustellen.
- Zum anderen erhält der Compiler die Vorschrift, wie die Bitfolge, die sich an dem Speicherplatz einer Variablen befindet, zu interpretieren ist (als ganze Zahl mit oder ohne Vorzeichen, als Gleitkommazahl, usw.).

Die Vereinbarung einer Variablen bewirkt im Allgemeinen ohne weiteres nicht gleichzeitig einen wohldefinierten Anfangswert für die Variable. Erforderlichenfalls kann man dies aber sicherstellen, indem man die Vereinbarung um ein Gleichheitszeichen und den gewünschten Wert erweitert. Diese Form der Vereinbarung heißt *Initialisierung*. Beispiel:

```
double x = 2.5e-7;  
enum farbe kreide = ROT;  
int i, summe = 0, quotient = 1, j;
```

Anfangswerte müssen immer individuell zugewiesen werden; eine abkürzende Schreibweise, um mehreren Variablen denselben Anfangswert zuzuweisen, gibt es nicht.

2.10 Namen

Um verschiedene Speicherplätze und andere Objekte, die in C-Programmen eine Rolle spielen, ansprechen zu können, muss man sie mit einem *Namen* oder *Bezeichner* versehen. Dies haben wir schon in verschiedenen Zusammenhängen kennengelernt. Entsprechend wird es Zeit, dass wir uns mit den Regeln für sie beschäftigen. Zur Bildung von Namen stehen folgende Zeichen zur Verfügung:

- uneingeschränkt die 52 Buchstaben, wobei zwischen Groß- und Kleinbuchstaben strikt unterschieden wird,
- die 10 Ziffern, wobei das erste Zeichen eines Namen keine Ziffer sein darf, um eine Unterscheidung von numerischen Konstanten zu erlauben, und
- das Unterstreichungszeichen (`_`). Dieses Zeichen darf auch am Anfang eines Namens stehen; sollte an dieser Stelle aber für Bibliotheksfunktionen reserviert bleiben.

Namen dürfen zwar beliebig lang sein, jedoch schreibt der Standard nur vor, dass die ersten 31 Zeichen *signifikant* sein müssen. Ein Compiler darf also Namen als gleich interpretieren, die sich erst im 32. oder einem späteren Zeichen unterscheiden. Die Minimalschranke für die Anzahl signifikanter Zeichen von Namen, die über eine Quelldatei hinaus verwendet werden sollen, liegt sogar nur bei 6.

Bestimmte Zeichenfolgen, die *Schlüsselwörter* (Tabelle 2.3), sind zwar wie Namen aufgebaut, dürfen aber nur mit ihrer festgelegten Bedeutung verwendet werden.

<code>auto</code>	<code>double</code>	<code>int</code>	<code>struct</code>
<code>break</code>	<code>else</code>	<code>long</code>	<code>switch</code>
<code>case</code>	<code>enum</code>	<code>register</code>	<code>typedef</code>
<code>char</code>	<code>extern</code>	<code>return</code>	<code>union</code>
<code>const</code>	<code>float</code>	<code>short</code>	<code>unsigned</code>
<code>continue</code>	<code>for</code>	<code>signed</code>	<code>void</code>
<code>default</code>	<code>goto</code>	<code>sizeof</code>	<code>volatile</code>
<code>do</code>	<code>if</code>	<code>static</code>	<code>while</code>

Tabelle 2.3: Schlüsselwörter von C

Da auch bei den Schlüsselwörtern zwischen Groß- und Kleinbuchstaben unterschieden wird, könnte man zum Beispiel zwar `Int` oder `INT` als Namen deklarieren, nicht jedoch `int`. Es ist in C üblich, wenn auch keineswegs vorgeschrieben, für die Namen von Variablen *nur* Kleinbuchstaben und für benannte Konstanten *nur* Großbuchstaben zu verwenden.

Um die Verständlichkeit von Quelltexten zu erhöhen, sollte man bei der Namensvergabe gewissen Konventionen folgen. Dies ist umso wichtiger, je umfangreicher der Quelltext ist. In der Regel sollte man die Namen von Variablen und Konstanten so wählen, dass sie etwas über die Bedeutung ihres Wertes aussagen. Das impliziert, dass Namen häufig 4 bis 6 Zeichen lang sind, oft auch länger. Die kleinen Beispiele in diesem Text können dies nicht hinreichend illustrieren.

In der Regel findet man in gut formulierten Quelltexten nur selten Variablennamen, die nur aus ein oder zwei Buchstaben bestehen. Sinnvoll sind solche kurzen Bezeichner allerdings in folgenden Fällen:

- Bei der Realisierung mathematischer Formeln sollte man die Bezeichnungen aus den Formeln übernehmen, soweit das möglich ist.
- *Hochgradig lokale* Variablen, etwa Zählvariablen in Schleifen, haben häufig keine benennbare Bedeutung. Übliche Namen für solche Variablen sind zum Beispiel

<code>i</code> , <code>j</code> , <code>k</code>	(Lauf-)Indizes
<code>m</code> , <code>n</code>	Anzahlen
<code>c</code> , <code>ch</code>	Zeichen
<code>s</code>	Zeichenketten („Strings“)
<code>p</code> , <code>z</code>	Zeiger

Man sollte auch daran denken, dass die Länge eines Namen nicht allein seligmachend ist. Zumindest für den Geübten hat der Name `i` *ebenso viel* (oder wenig) Aussagekraft wie der Name `laufvariable`!

Und noch etwas sollte man beachten:

- Zu ähnliche Namen verleiten zu Schreib- und Lesefehlern.
- Die Ziffer 1 und die Buchstaben I und l verwechselt man besonders leicht, ebenso die Ziffer 0 und den Buchstaben O (und auch den Buchstaben Q).

2.11 Typnamen

Das Schlüsselwort `typedef` erlaubt die Deklaration eigener Typnamen, die dann im weiteren Programm wie die entsprechenden Schlüsselwörter verwendet werden können:

```
typedef typ name;
```

Üblich ist es, für die Namen nur Großbuchstaben zu verwenden oder ihnen `_t` anzufügen. Diese beiden Varianten werden auch von der Standardbibliothek verwendet. Beispiel:

```
typedef int  INTEGER;
typedef enum farbe farbe_t;

INTEGER i = 1;
farbe_t kreide;
```

Wie man sieht, werden durch `typedef` keine neuen Typen deklariert, sondern nur bekannte Typen benannt. Entsprechend bringt `typedef` an dieser Stelle nichts wirklich Neues, auch wenn man die positiven Auswirkungen auf die Lesbarkeit der Programme nicht unterschätzen sollte. In der Standardbibliothek werden an mehreren Stellen `typedef`-Deklarationen verwendet. Beispiele findet man in Abschnitt 4.14 oder 9.6.

